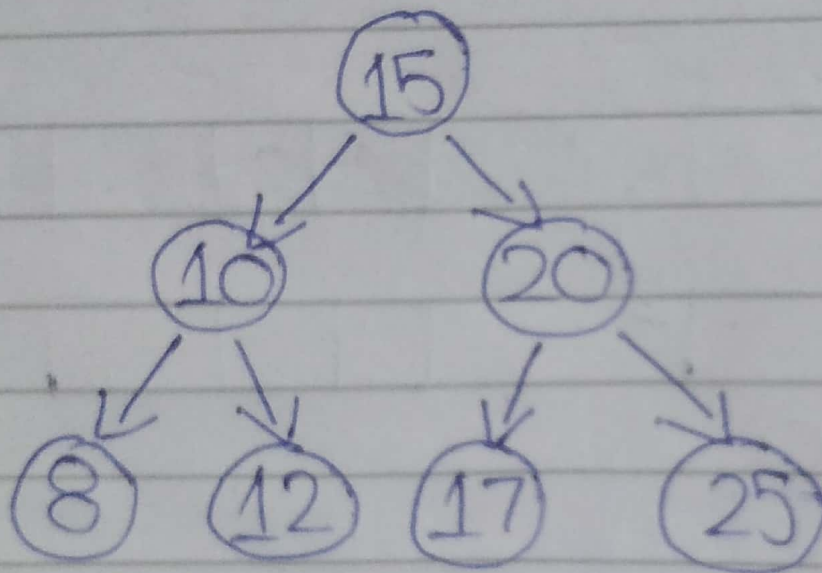


Binary Search Tree (BST)

A binary tree in which for each node, value of all the nodes in left subtree is lesser or equal and value of all the nodes in right subtree is greater.



Bst

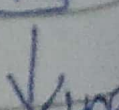
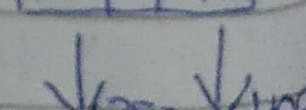
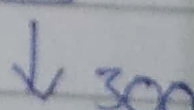
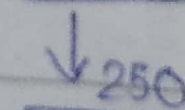
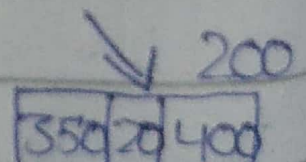
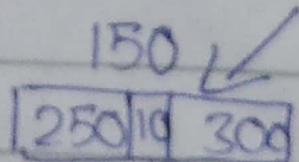
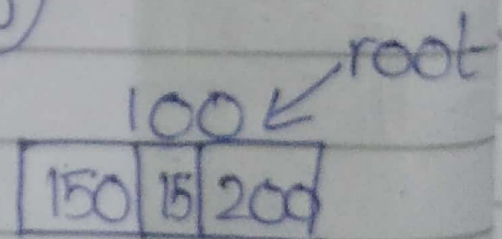
```
struct tNode {
```

```
    int data;
```

```
    Node* left;
```

```
    Node* right;
```

```
};
```



```
BstNode* root;  
root = NULL;
```

```
int main() {
```

```
    BstNode* root = NULL;
```

```
    Insert(root, 15);
```

```
    Insert(root, 10);
```

```
    Insert(root, 20);
```

```
}
```

```
BstNodeNode* GetNewNode  
(int data) {
```

```
    BstNode* newNode = new  
    BstNode();
```

```
    newNode->data = data;
```

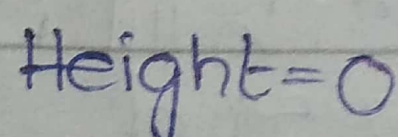
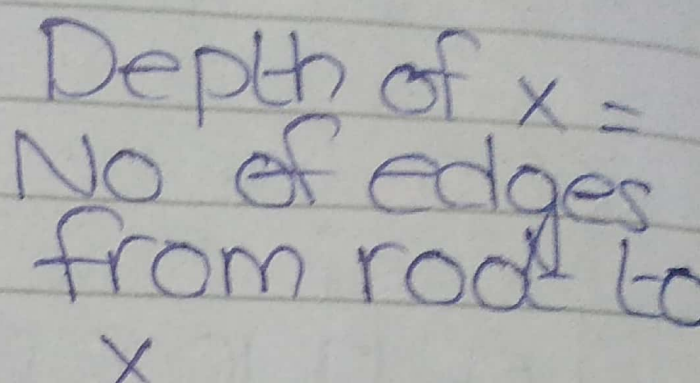
```
    newNode->left = newNode->  
    right = NULL;
```

```
    return newNode;
```

```
}
```

```
Bst-Node* Insert(Bst-Node*  
root, int data)
```

```
{  
    if (root == NULL)  
    {  
        root = GetNewNode(data);  
    }  
    else if (data ≤ root → data)  
    {  
        root → left = Insert(root  
→ left, data);  
    }  
    else  
    {  
        root → right = Insert (root  
→ right, data);  
    }  
    return root;  
}
```



in longest distance from x to leaf.

Binary Tree:-

A tree in which each node have at most two children.

Proper Binary Tree:-

Each node can have either two or zero children.

Complete Binary Tree:-

All levels except the last are completely filled and all nodes are as left as possible.

Perfect Binary Tree:-

All levels will be completed till the very end.

```
int main() {
```

```
    BstNode* root = NULL;  
    Insert(root, 15);  
    Insert(root, 10);  
    Insert(root, 20);
```

```
int num;
```

```
cout << "Enter no to be  
searched : \n";  
cin >> num;
```

```
if(search(root, num) == true)  
    cout << "found";  
else  
    cout << "not found";
```

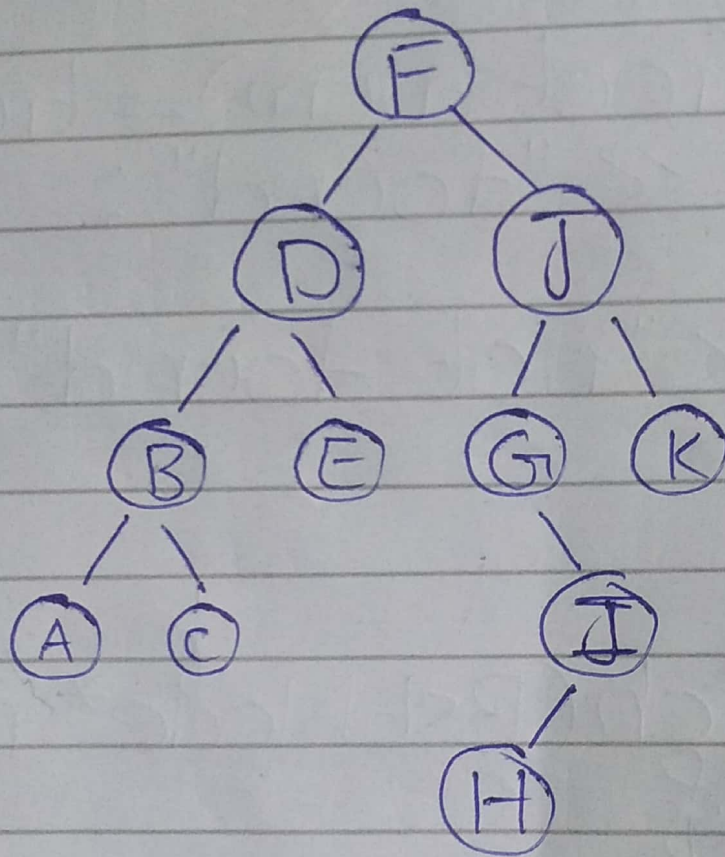
```
}
```

```
bool search(BstNode* root  
int data){
```

```
    if(root == NULL)  
        return false;  
    else if(root->data == data)  
        return true;  
    else if(data <= root->data)  
        return search(root->left,  
data);  
    else  
        return search(root->right,  
data);
```

Binary Tree Traversal:-

Process of visiting each node in the tree exactly once in some order.



*Pre Order:- (Root, Left, Right)

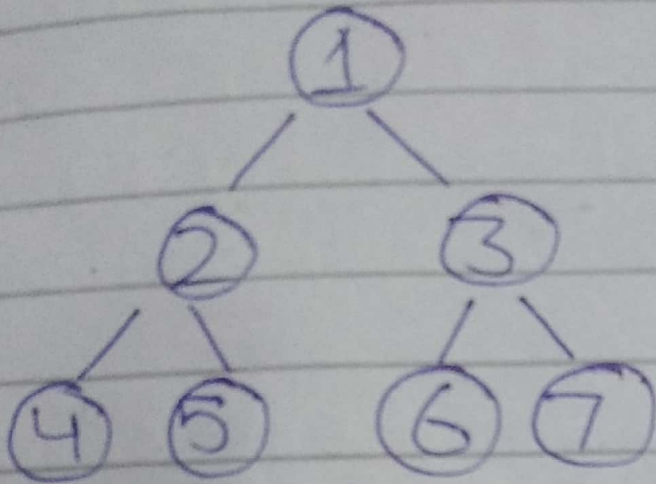
F D B A C E J G I H K

*In Order:- (Left, Root, Right)

A B C D E F G H I J K

*Post Order (Left, Right, Root)

A C B E D H I G K J F



Pre Order:-

1 2 4 5 3 6 7

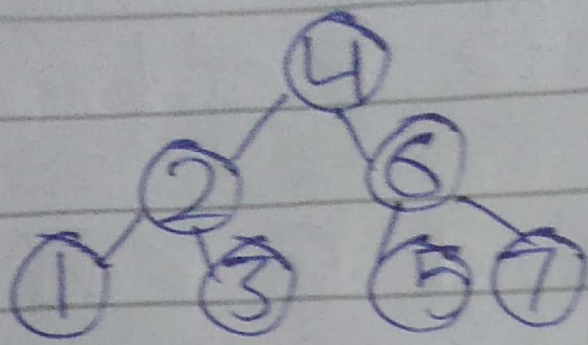
In Order:-

4 2 5 1 6 3 7

~~1 2 3 4 5 6 7~~

Post Order:-

4 5 2 6 7 3 1



Pre Order:-

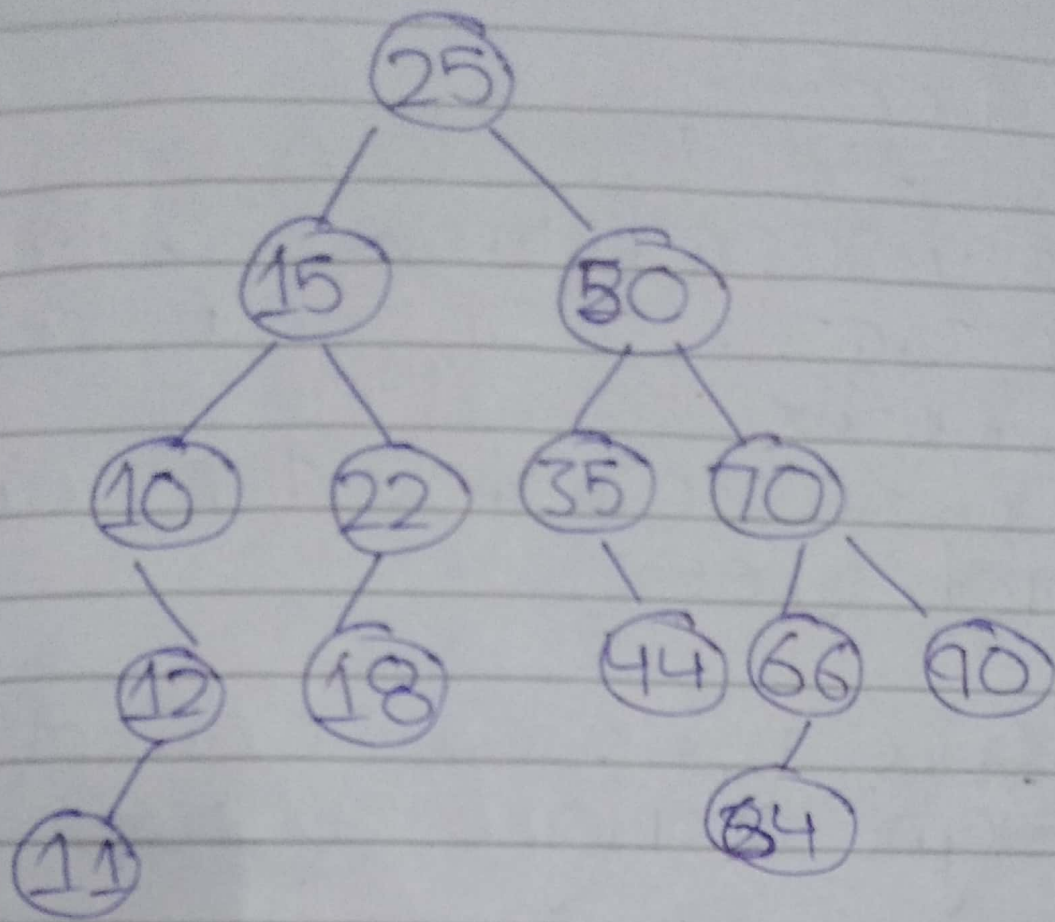
4 2 1 3 6 5 7

In Order:-

1 2 3 4 5 6 7

Post Order:-

1 3 2 5 7 6 4



Pre Order :-

25, 15, 10, 12, 11, 22, 18,
50, 35, 44, 70, 66, 64, 90

In Order :-

~~45, 25, 50, 10, 22, 11, 12, 10, 15,~~
~~22, 18, 10, 11, 12, 15, 18, 22, 25,~~
 35, 44, 50, 64, 66, 70, 90

Post Order :-

11, 12, 10, 18, 22, 15, 44, 35,
64, 66, 90, 70, 50, 25.

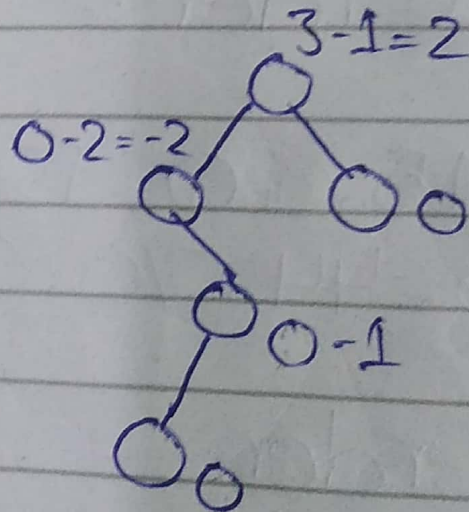
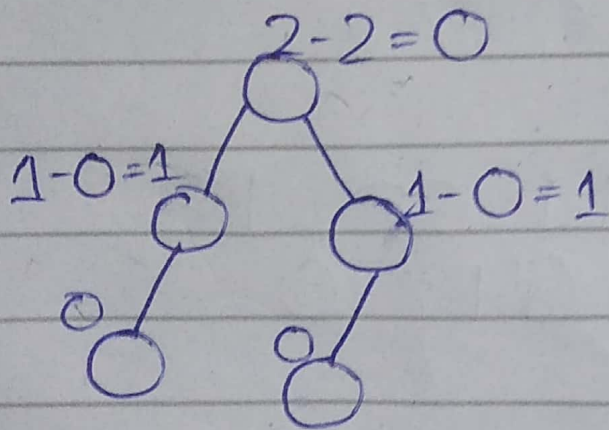
Na. _____

AVL Tree :-

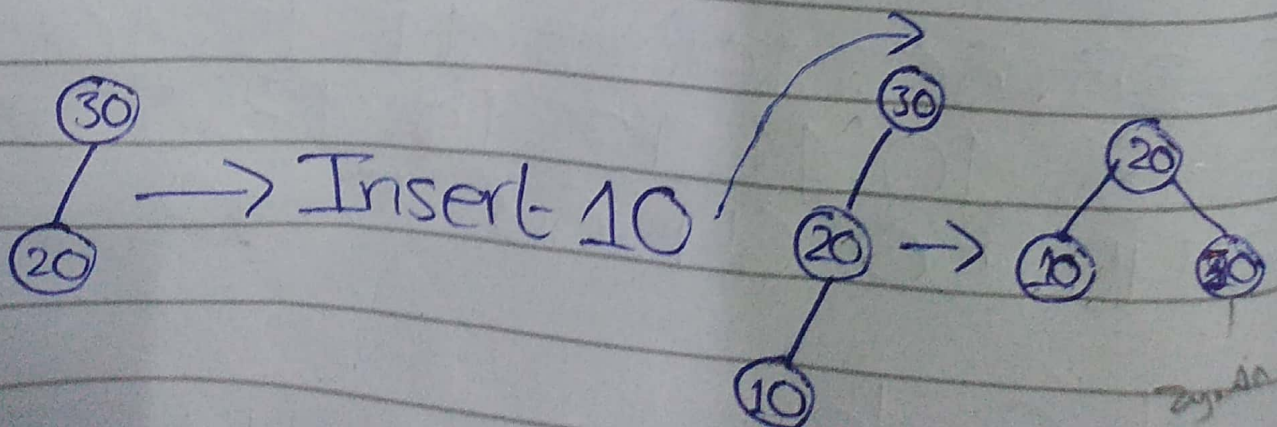
An AVL tree is a type of binary search tree. AVL trees have property of self balancing in addition to all other properties exhibited by binary search trees.

Balance factor = height of left - height of right subtree.

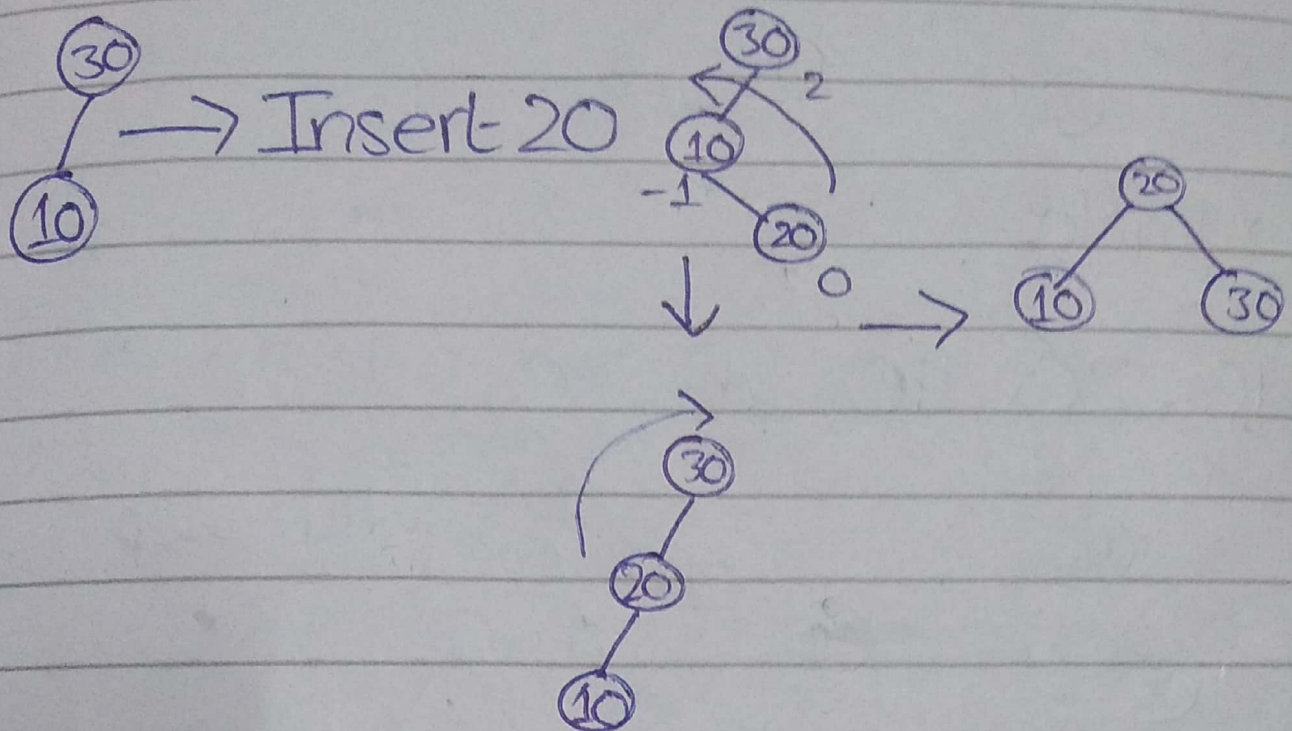
$$bf = hl - hr = \{-1, 0, 1\}$$



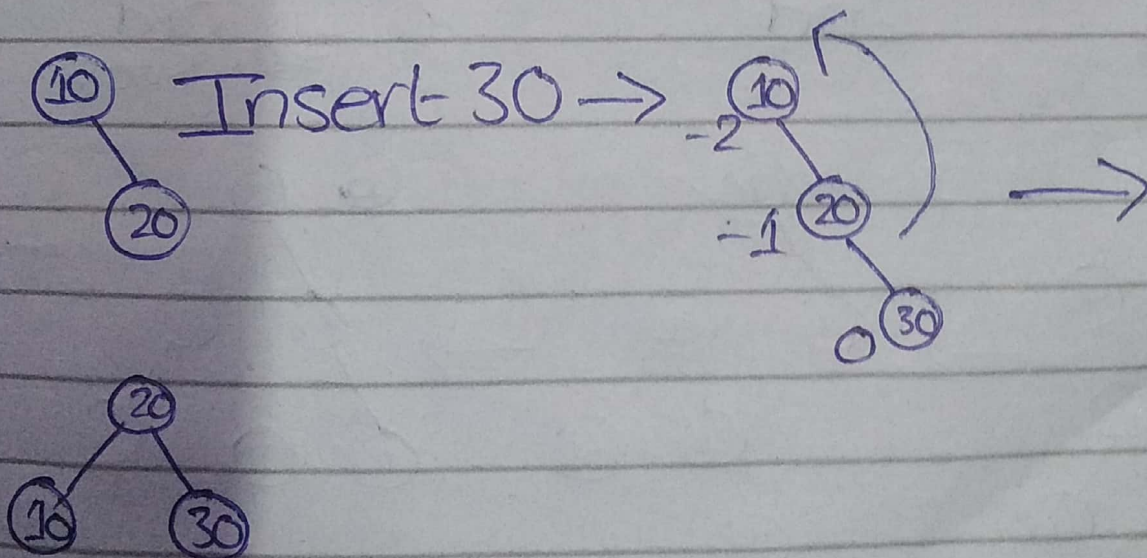
LL-Rotation :-



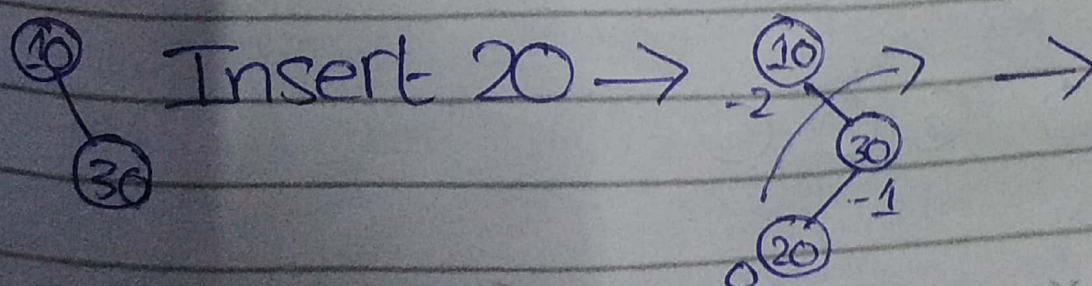
LR-Rotation:-



RR-Rotation:-



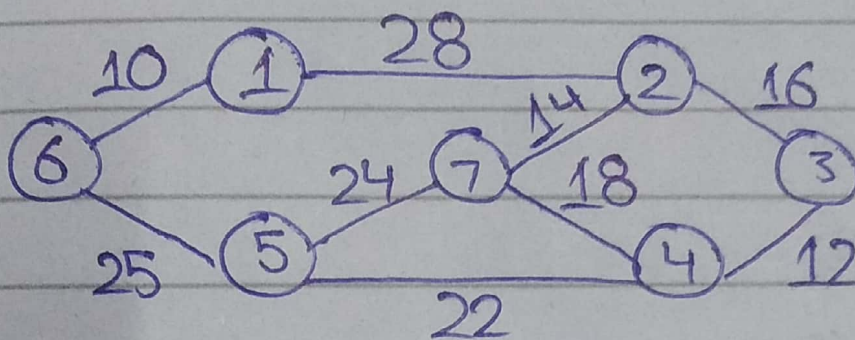
RL-Rotation:-



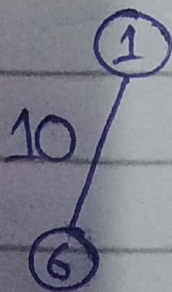
Q. 1 2 3 4 5 6 7 8

29-12-22

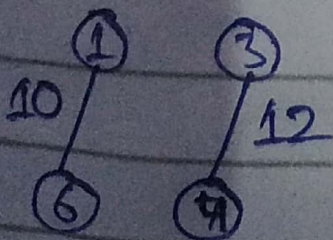
Minimum Spanning Tree Kruskal's Algorithm

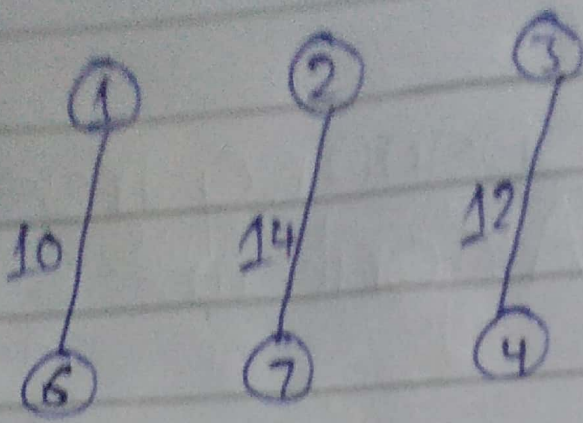


Step 1:

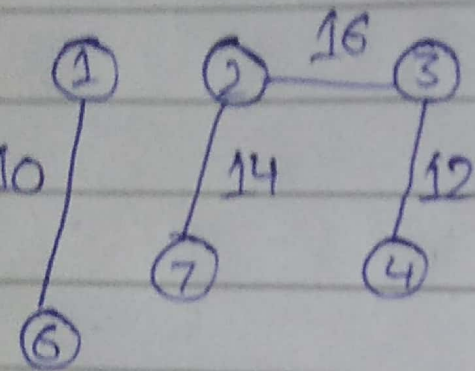


Step 2:

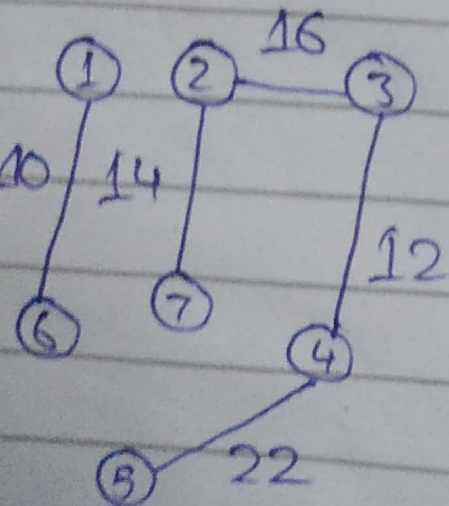




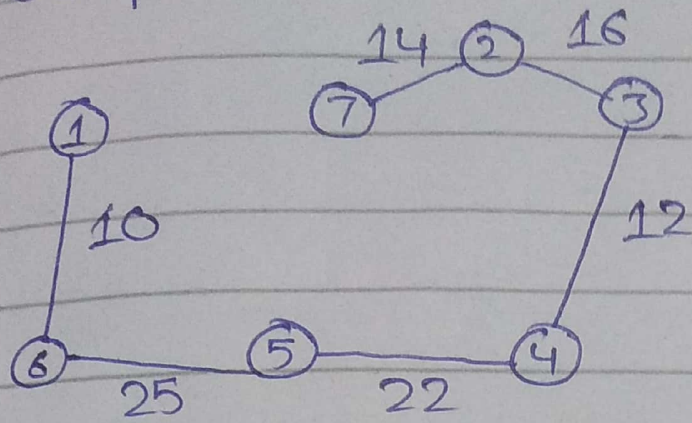
Step 4:-



Step 5:-



Step 6:-

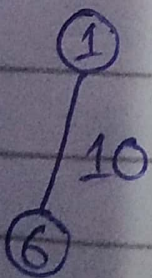


Weight of MST:-

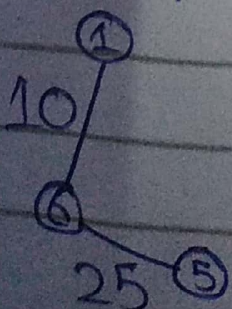
$$\begin{aligned} &= \text{Sum of all edges} \\ &= 10 + 25 + 22 + 12 + 16 + 14 \\ &= 99 \end{aligned}$$

Prim's Algorithm:-

Step 1:-



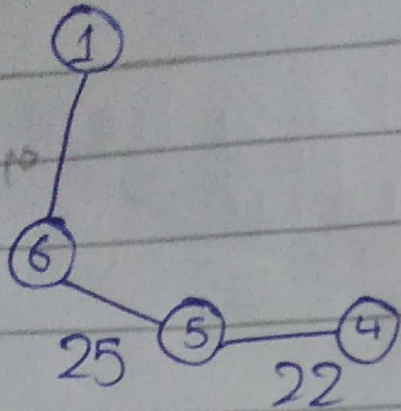
Step 2:-



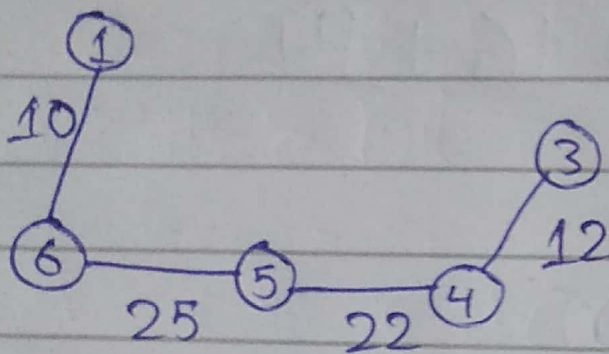
RG

No. _____

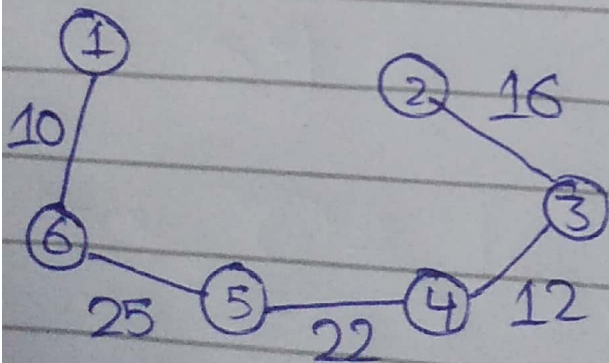
Step 3:-



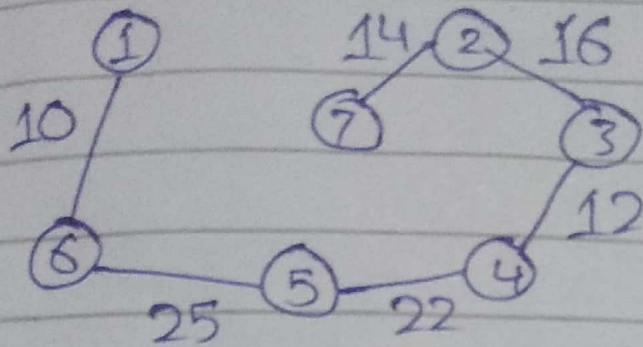
Step 4:-



Step 5:-



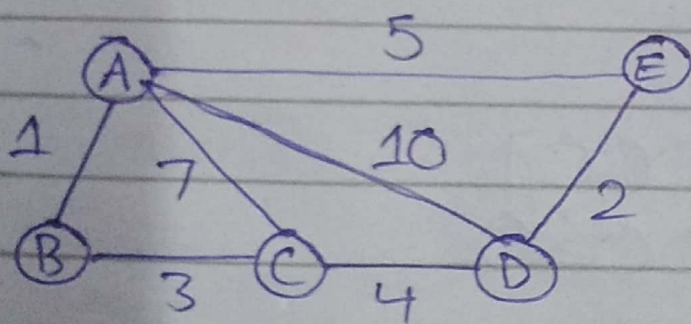
Step 6:-



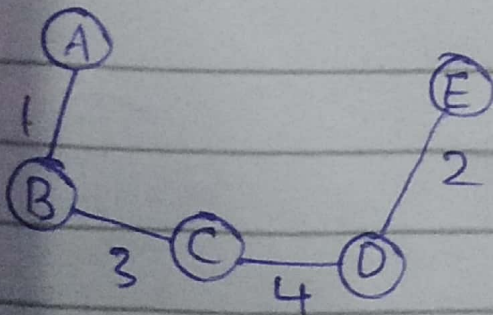
Weight of MST:-

= Sum of all edges

$$= 10 + 25 + 22 + 12 + 16 + 14 = 99$$

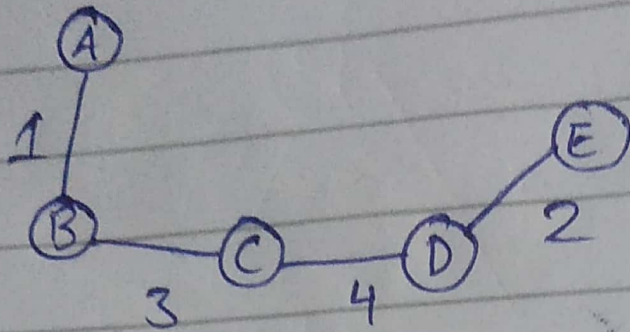


Kruskal's Algorithm:-



Weight of MST:- 10

Prim's Algorithm:-



Weight of MST :- 10

30-12-22

Huffman Coding

ASCII 8 bit

A	65	01000001
B	66	01000010
C	67	:
D	68	:
E	69	:

$$8 \times 20 = 160 \text{ bits}$$

Message: BCCA BBDDAECC
BBAE DDCC

Char	Count	Code
A	3	001
B	5	10
C	6	11
D	4	01
E	2	000
	20	12 bits

5x8 bits

$3 \times 3 = 9$

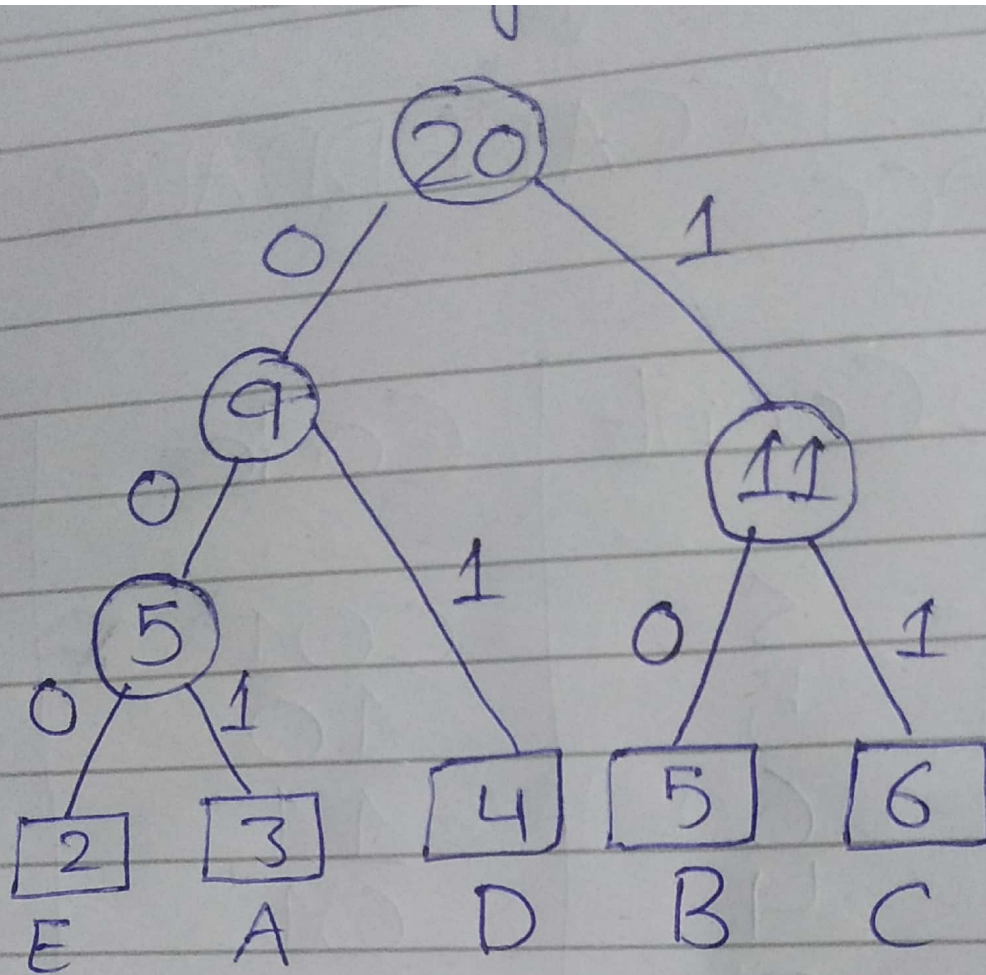
$5 \times 2 = 10$

$6 \times 2 = 12$

$4 \times 2 = 8$

$2 \times 3 = 6$

45 bits



Msg - 45 bits
Table - 52 bits
Total - 97 bits